

M1. Gestion des erreurs



Ce module du cours se concentre sur l'importance de la **gestion des erreurs dans la programmation en Python**. Vous découvrirez comment anticiper et gérer efficacement les erreurs qui peuvent survenir lors de l'exécution de votre code.

Au cours de ce module, vous apprendrez les bases de la gestion des erreurs en Python 3, y compris les types d'erreurs courantes et comment les détecter à l'aide des mécanismes de traitement des exceptions. Vous explorerez également les blocs try-except et découvrirez comment les utiliser pour capturer et traiter les erreurs de manière contrôlée.

De plus, vous apprendrez des techniques avancées pour la gestion des erreurs, telles que la création de vos propres exceptions personnalisées et la gestion des exceptions multiples. Vous comprendrez également l'importance de la manipulation des erreurs de manière appropriée afin d'améliorer la maintenance et la lisibilité de votre code.

En suivant ce module, vous serez capable de détecter les erreurs potentielles dans votre code et de mettre en place des mécanismes appropriés pour les gérer. Vous gagnerez en confiance dans la programmation Python et pourrez développer des applications plus robustes et fiables.

Prérequis : Ce cours suppose une connaissance de base de la programmation en Python 3, y compris les concepts de base tels que les variables, les structures de contrôle et les fonctions.

— Les erreurs courantes et les exceptions

— Gestion de exceptions avec try/except

— Levée d'exceptions avec raise

— Création de ses propres exceptions

— Review

Les erreurs courantes et les exceptions

Les erreurs courantes en Python

Beaucoup d'erreurs peuvent survenir durant l'exécution d'un *script* Python :

- Erreur de syntaxe.
- Erreur d'indentation du code.
- Erreur d'exécution.
- Etc.



L'utilisation des termes "**script** Python" et "**programme** Python" dépend de la nature et de la complexité du code Python en question. Les scripts Python ont tendance à être plus courts, plus simples et utilisés pour des tâches spécifiques, tandis que les programmes Python sont plus larges et plus complexes, avec potentiellement plusieurs fonctionnalités et modules intégrés.

Quelques exemples d'erreurs

Nous avons cité les erreurs les plus courantes, souvent liées à un défaut d'attention lors de la rédaction du code.

Voici des exemples qui illustrent ces erreurs.

1. L'erreur de syntaxe

Cette erreur survient lorsqu'on ne **respecte pas les règles de syntaxe du langage**.

```
monNombre = 42  
print('monNombre'
```

Code comportant une erreur de syntaxe : il manque la parenthèse fermante

Ce code vous renverra le message suivant lors de l'exécution :

```
SyntaxError: '(' was never closed
```

Message d'erreur dans le cas d'une erreur de syntaxe.

En français, "Erreur de syntaxe : '(' n'a jamais été fermé"

2. L'erreur de nom

Cette erreur survient lorsqu'on fait **référence à une variable ou une méthode qui n'existe pas**.

```
monNombre = 42  
print(monnombre)
```

Code comportant une erreur de nom

Ce code vous renverra le message suivant lors de l'exécution :

```
NameError: name 'monnombre' is not defined. Did you mean: 'monNombre'?
```

Message d'erreur renvoyé dans le cadre d'une erreur de nom.

En français, "Erreur de nom : le nom 'monnombre' n'est pas défini. Vouliez-vous utiliser 'monNombre' ?"

En Python, comme dans de nombreux langages, le nom des variables est **sensible à la casse**.

Dans notre code, on essaie d'afficher le contenu de la variable **monnombre** mais c'est une variable **monNombre** qui a été créée.

monNombre et **monnombre** sont deux variables distinctes !

 La **casse** en programmation fait référence à la distinction entre les lettres majuscules (ou uppercase) et les lettres minuscules (ou lowercase) dans l'écriture d'identifiants, tels que les noms de variables, de fonctions, de classes, etc.

3. L'erreur d'index

Cette erreur survient lorsqu'on cherche à extraire la valeur d'une séquence (telle qu'une liste ou une chaîne de caractères) à une position qui se situe en dehors de cette séquence.

```
monTexte = "Hello All"      #Ce texte fait 9 caractères
print(monTexte[9])          #Nous cherchons à afficher le 10ème caractère
```

Code comportant une erreur d'index

Ce code renverra le message suivant lors d'une tentative d'exécution :

```
IndexError: string index out of range
```

Message d'erreur affiché lors d'une erreur d'index.

En français, "Erreur d'index : index de chaîne de caractères hors-limites"

En Python une chaîne de 9 caractères est traitée comme une suite de caractères indexés de 0 à 8.

L'utilisation de l'index 9 fait donc référence au 10e caractère d'une chaîne... qui n'en comporte que 9 !

4. L'erreur d'indentation

Cette erreur survient lorsqu'on ne respecte pas les règles d'*indentation* du langage Python.

Attention : un problème d'indentation n'empêche pas toujours un programme de s'exécuter. Un programme peut être mal indenté logiquement mais respecter les règles d'indentation.

 En programmation, l'**indentation** fait référence à la mise en retrait du code source pour délimiter les blocs de code et améliorer la lisibilité. Cela consiste généralement à ajouter des espaces ou des tabulations au début des lignes de code pour créer une structure visuelle.

```
#CAS 1
monTexte = "Hello All" #ce texte fait 9 caractères

if monTexte[8] == 'e':
    print("La dernière lettre est 'e'")
    print("La dernière lettre est celle que nous attendions")
```

```
#CAS 2
monTexte = "Hello All" #ce texte fait 9 caractères

if monTexte[8] == 'e':
    print("La dernière lettre est 'e'")
print("Heureusement que c'est un 'e' d'ailleurs")
```

Code comportant des erreurs d'indentation

Dans le **premier cas**, le code renvoie une erreur d'indentation :

```
IndentationError: unexpected indent
```

Message d'erreur d'indentation.

En français : "Erreur d'indentation : indentation inattendue".

Effectivement, le langage ne comprend pas pourquoi le second print est indenté par rapport au premier : ça n'a pas de sens.

Dans le **deuxième cas**, Python ne constate pas d'erreur d'indentation. Le code fonctionnera sans aucun problème. Cependant, il ne fonctionnera pas comme attendu car le second print devrait faire partie du bloc if. Il s'agit ici d'une erreur logique d'indentation qui ne sera pas signalée par le langage car ce dernier ne peut deviner quelle logique nous suivons.

Il existe encore bien d'**autres erreurs** telles que :

- Les **erreurs de fichiers**: fichier n'existant pas, impossible à lire / à créer / à modifier.
- La traditionnelle **erreur de division par zéro**.
- Les **erreurs d'importation de module** : importation de modules inexistants / pas installés / mal orthographiés.

Ces erreurs, lorsqu'elles sont liées à l'exécution du code et non à sa bonne rédaction sont appelées des **exceptions**.

Ces exceptions peuvent être gérées par le développeur dans son script.

Ainsi, le script ne s'arrêtera pas brutalement mais exécutera un code approprié de gestion de l'erreur (afficher un message d'erreur personnalisé, demander à l'utilisateur d'effectuer une action, recommencer un traitement, etc.).

CONTINUER

Gestion de exceptions avec try/except

Comme expliqué précédemment, il est possible de **prévoir les exceptions** et donc d'**intégrer un traitement alternatif à notre code**.

```
def divide(nb1, nb2):  
    result = nb1/nb2  
    return result  
  
print(divide(4,0))
```

Exemple de code qui va effectuer une division par zéro... et donc générer une exception.

Dans ce code, une fonction **divide** est définie. Elle accepte deux paramètres : **nb1** et **nb2**. La fonction renvoie le résultat de la division de **nb1** par **nb2**.

Cette méthode fait abstraction de tout risque potentiel de division par zéro. C'est pourtant justement ce que nous cherchons à faire en renseignant les nombres 4 et 0 comme paramètres.

L'exécution du code va donc lever l'exception suivante :

```
ZeroDivisionError: division by zero
```

Message d'erreur affiché lors d'une exception générée par une division par zéro.

En anglais : "Erreur de division par zéro".

Utilisation du bloc try....except...

Les commandes **try:** (en anglais, essayer) et **except:** (en anglais, exception) permettent d'**encadrer un code qui risque de lever une exception et de proposer un traitement alternatif.**

Ainsi, en cas d'exception, aucun message d'erreur ne sera affiché mais le code contenu dans le bloc except sera exécuté.

```
try:
    code qui risque de lever une exception
except:
    code a exécuter si une exception a été levée
```

Dans le cas où une exception survient dans le bloc de code **try:**, l'**exécution du bloc s'interrompt à la ligne où l'erreur survient, le reste du code du bloc try: n'est pas exécuté** et **le code du bloc except: est exécuté.**



Il est inutile d'utiliser un try...except... dans un bloc de code où aucune exception n'est susceptible d'être levée.

Adaptons notre exemple précédent:

```
def divide(nb1, nb2):  
    try:  
        result = nb1/nb2  
        return result  
    except:  
        return float('inf')  
  
print("Le resultat de la division est : ",divide(4,0))
```

Code prenant en compte une possible division par zéro.

Dans cette version de notre fonction de division, le cas de la division par zéro est **anticipé par la mise en place d'un bloc d'exception**.

Dans le bloc **try:** nous insérons le code qui risque de lever une exception (`result = nb1/nb2`).

Dans le bloc **except:**, nous prévoyons de renvoyer la valeur `float('inf')`.



En Python 3, `float('inf')` permet de renvoyer la notion d'infini.

```
Le resultat de la division est :  inf
```

Affichage après exécution du script

L'exception est gérée et le code continue de tourner comme attendu.

Ajout du bloc finally...

Le bloc try...except... **peut être suivi d'un bloc facultatif additionnel finally..**

Ce bloc de code sera **exécuté dans tous les cas !**

Il permet de globaliser un traitement plutôt que de le dupliquer dans le bloc try: et dans le bloc except:.

Le code de notre fonction (et son utilisation) peut être amélioré de la manière suivante :

```
def divide(nb1, nb2):  
    result = float('inf')  
    try:  
        result = nb1/nb2  
    except:  
        print("Il n'est pas possible de diviser par zéro")  
    finally:  
        return result  
  
print("Le resultat de la division est : ",divide(4,0))
```

Code mettant en œuvre des blocs `try`, `except` et `finally`.

1

Dans la fonction **divide**, la variable **result** est initialisée avec l'infini.

2

Le code **result = nb1/nb2** est placé dans un bloc **try**: car il y a un risque de division par zéro et donc un risque d'exception.

3

En cas de division par zéro (bloc **except**), le texte "Il n'est pas possible de diviser par zéro" est affiché à l'écran.

4

Dans tous les cas, le contenu de la variable **result** est renvoyé par la fonction **divide**.

CONTINUER

Levée d'exceptions avec raise

Il peut être intéressant de pouvoir **lever manuellement des exceptions**.

On pourra, par exemple, afficher un message personnalisé dans le cas d'une *méthode* qui n'est pas encore implémentée ou qui est dépréciée.

 Une **méthode** est une fonction liée à un objet en programmation orientée objet. Tout cela sera expliqué dans les prochains modules.

Ce mécanisme sera généralement utilisé en phase de développement.

L'exemple typique d'erreur que nous pouvons lever dans ce genre de situation serait **`NotImplementedError()`**.

La commande **raise** doit être utilisée et suivie de l'exception à lever et de l'éventuel message à renvoyer dans la console de débogage.

Reprenons le cas de notre fonction **divide** :

```
def divide(nb1, nb2):  
    raise NotImplementedError("This method should be implemented First")  
  
print("Le resultat de la division est : ",divide(4,0))
```

Code levant manuellement une erreur d'implémentation (absence).

Le résultat de l'appel de la fonction **divide** sera donc le suivant :

```
NotImplementedError: This method should be implemented First
```

Message d'erreur affiché dans la console de débogage.

Alors que le système ne prévoit pas de levée d'exception, nous en avons ajouté une manuellement afin que, quoi qu'il puisse se passer dans le code, une exception soit levée lors de l'appel de la fonction `divide`.

CONTINUER

Création de ses propres exceptions

i **Attention: cette section traite de programmation Orienté Objet qui sera vue en détails plus loin dans ce cours. N'hésitez pas à revenir dans cette section une fois la programmation orientée objet abordée.**

Le langage Python 3 lève nativement énormément d'exceptions et il est possible de les lever soi-même au moyen de la commande `raise`.

En Python, la majeure partie des exceptions dérivent d'un conteneur principal (une *classe*) appelé **Exception**.

i Une classe en Python 3 est un modèle ou un plan à partir duquel on peut créer des objets.

Pour créer une exception, il faut utiliser ce conteneur principal.

```

class ShouldWorkMoreException(Exception):
    def __init__(self, message, code):
        super().__init__(message)
        self.code = code

    def errorCode(self):
        return this.code

    def __str__(self):
        return f"ShouldWorkMoreException thrown with message '{super().__str__}'"

def divide(nb1, nb2):
    raise ShouldWorkMoreException("This method should be implemented First")

try:
    print("Le resultat de la division est : ",divide(4,0))

except ShouldWorkMoreException as e:
    print(e)

```

Code dans lequel on trouve la création d'une exception utilisateur.

Dans ce code, nous définissons une classe **ShouldWorkMoreException** qui utilise la classe de base **Exception** et dans laquelle nous ajoutons une nouveauté : un code erreur.

Ce code erreur peut être utilisé pour indiquer le comportement à adopter en fonction du code erreur renvoyé.

Lors de l'appel de la fonction **divide**, l'erreur est levée en précisant un code 500 et une structure try....except standard est utilisée pour traiter l'erreur.

Rien de bien compliqué en soit, même si je suis persuadé que de prime abord ce code doit tout bonnement vous effrayer.

Pas de panique tout s'éclaircira dans quelques modules 😊😊😊

Gérer les exceptions multiples

Au delà de l'aspect orienté objet du code précédent, vous avez probablement constaté que l'exception était traitée également de manière différente.

De fait, il est tout à fait possible de **manipuler l'exception qui est retournée en l'assignant à une variable** ('e' dans l'exemple).

Encore plus intéressant, il est possible de gérer les exceptions différemment en fonction du type d'exception qui est levée. Ainsi dans le cas d'une exception liée au traitement d'un fichier, par exemple, il est possible de différencier le traitement qui sera effectué en fonction du type d'exception (problème de lecture, problème d'écriture, problème de création, inexistence du fichier, etc.).

Exemple :

```
def divide(nb1, nb2):
    try:
        return nb1/nb2
    except ZeroDivisionError:
        return "Erreur de division par zéro"
    except TypeError:
        return "Les types utilisés ne correspondent pas avec l'opération"

print("Le resultat de l'opération est :", divide(4,0))
print("Le resultat de l'opération est :", divide(4,"toto"))
```

Code qui gère plusieurs type d'exception.

1

Le premier print sera traité par l'exception de division par zéro car on passe les valeurs 4 et 0 à la fonction `divide` qui doit diviser la première valeur par la seconde. Le premier bloc `except:` sera donc exécuté.

2

Le second print ne sera pas traité par l'exception de division par zéro car il n'y a pas de division par zéro. Par contre, le second `except:` sera exécuté car il y a bien une exception de type `TypeError`.

```
Le resultat de l'opération est : Erreur de division par zéro.  
Le resultat de l'opération est : Les types utilisés ne correspondent pas à
```

Affichage après exécution du code.



Si vous gérez des erreurs particulières et qu'une erreur non couverte par votre code survient, celle-ci ne sera pas gérée et votre code en subira les conséquences. Veillez à penser à toutes les exceptions qui peuvent se produire !

CONTINUER

Review

À quoi correspond généralement l'erreur de type `NameError` ?

- ☐ À une erreur qui survient lorsqu'il est fait référence à une variable ou à une méthode qui n'existe pas.
- ☐ À une erreur qui survient lorsqu'on essaie d'extraire la valeur d'une séquence à une position qui se situe en dehors de cette séquence.
- ☐ À une erreur qui survient lors d'un problème de nommage d'un script Python.

SUBMIT

Vrai ou faux : Python nous impose un ensemble déterminé d'erreurs avec lesquelles nous devons composer?

☐

Vrai

☐

Faux

SUBMIT

Remettez de l'ordre dans ces définitions

SUBMIT

Complète la phrase suivante :

Pour lever des exceptions utilisateur, il faut utiliser l'instruction

-----()

Tapez votre réponse ici

SUBMIT

CONTINUER

Exercice

Écrivez un code qui demande l'âge de l'utilisateur jusqu'à ce que celui-ci fournisse une valeur entière correcte comprise entre 1 et 99 (ces deux nombres inclus).

Gérez les exceptions de type.

Exemple d'exécution:

```
Give your age : trentre-trois
Give your age in a numerical integer format
Give your age : 100
Give a value between 1 and 99 included
Give your age : 55
Given age is : 55
```

Exemple d'exécution du script.

